

Experiment 6 Timers & Event-Driven State

ART
9/28/20

Purpose

Build a traffic signal controller that uses timed phases & car detection inputs to decide when traffic should continue in the current direction

References

- RCET 3375 Lab Standard
- RCET PIC-As style guide
- RCET 3373 PIC16F883 Timers
- RCET Flowchart Guide
- PIC Datasheet & Family manual
- Previous Lab Scans

Part 1 - 20ms Timer Proof of Life

Goal

Configure one PIC16F883 hardware timer to generate a periodic interrupt every 20ms. For every timer interrupt, service the timer, & drive a diagnostic output high & immediately low

Before Lab

Prepare:

- Timer choice & reasoning
- Complete timer calculation
- timer & interrupt SFR documentation
- Timer - Period boundary definition
- Predicted period
- Predicted Tcy offset from the timer period boundary to the diagnostic rising edge
- Selected port A pins
- Flowchart & source code

* For this Part I chose timer 2. Timer 0, while it is the simplest timer to build, has no way to hit 20,000µs perfectly. The closest you can get is 19968µs. Timer 1 holds 2 registers for a count so it has the highest count potential. Timer 2 simply adds a postscaler, effectively doubling Timer 0's count & the inclusion of PR2 makes deciding a specific count far easier than 0 or 1.

TABLE 14-1 SUMMARY OF ASSOCIATED TIMER2 REGISTERS

Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Register on Page
INTCON	GIE	PEIE	TOIE	INTE	RBIE	TOIF	INTF	RBIF	32
PIE1	—	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE	33
PIR1	—	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF	35
PR2	Timer2 Module Period Register								83
TMR2	Holding Register for the 8-bit TMR2 Register								83
T2CON	—	TOUTPS3	TOUTPS2	TOUTPS1	TOUTPS0	TMR2ON	T2CKPS1	T2CKPS0	84

Legend: x = unknown, u = unchanged, — = unimplemented read as '0'. Shaded cells are not used for Timer2 module.

ART
9/24/20

Experiment 6 Timers & Event-Driven State

ART
9/29/26

$$1\mu s \cdot \text{Prescaler} \cdot (PR2+1) \cdot \text{Postscaler} = 20,000$$

$$1+4+16$$

the detection of a match takes an instruction cycle.

$$1-16$$

*

$$1\mu s \cdot 16 \cdot (PR2+1) \cdot 5 = 20,000$$

PR2 = 249 * Three part Equation like 19 165 used AI to streamline

must be 16

Because 20,000 is impossible otherwise. $\rightarrow 4 \cdot 255 \cdot 16 = 16,320$

7

Setup:

```
; add device setup here
BSF 0x03, 06 ;
BSF 0x03, 05 ; Set bank 3 active
CLRF 0x189 ; ANSEL analog/digital select portB pins as digital
CLRF 0x188 ; ANSEL analog/digital select portA pins as digital
```

```
BSF 0x03, 06 ;
BCF 0x03, 5 ; Set bank 2 active
BSF 0x09, 1 ; Turn TiG-not function off
```

```
BCF 0x03, 06 ;
BSF 0x03, 05 ; Set bank 1 active
CLRF 0x85 ; TrisA sets all portA pins as outputs
MOVLW 0x01 ; Make W all highs
MOVWF 0x86 ; Move W to TrisB
CLRF 0x87 ; TrisC sets all portC pins as outputs
BSF 0x0C, 1 ; PIE1 Enable Timer2 interrupt
MOVLW 0xF9 ; Compare limit set at
MOVWF 0x92 ; Set the Timer2 Compare limit
```

```
BCF 0x03, 06 ;
BCF 0x03, 05 ; Set bank 0 active
CLRF 0x06 ; clear port B
CLRF 0x05 ; clear port A
CLRF 0x07 ; Clear portC
MOVLW 0b00100011; Put binary register in W
MOVWF 0x12 ; Turn TMR2 Postscaler to 1:16 and prescaler 1:16
```

```
; Interrupt setup
CLRF 0x0B ; Clear INTOCN to disable all interrupts by default
BSF 0x12, 2 ; Make TMR2 begin counting
BSF 0x0B, 6 ; Enable Peripheral Interrupts
BSF 0x0B, 7 ; Enable Global interrupts
goto Main
```

Main:

```
MOVLW 0x01 ; Put bit 0 high in W
XORWF 0x05 ; Toggle bit 0 in PortA
goto Main
```

ISRHandler:

```
MOVWF W_TEMP ; Copy W to TEMP register
SWAPF STATUS, 0 ; Swap status to be saved into W
; Swaps are used because they don't change the status bits
MOVWF STATUS_TEMP ; Save status to bank zero STATUS_TEMP register
BTFSC PIR1, 1 ; Test the TMR2 PR2 match flag bit (must be in Bank0)
goto Timer2Service
goto LeaveHandler
```

Timer2Service:

```
CLRF TMR2 ; Clear the timer2 count
BSF PORTA, 1 ; Enable Diagnostic pin
BCF PORTA, 1 ; Disable Diagnostic pin
BCF PIR1, 1 ; Clear Timer2 flag
goto LeaveHandler
```

LeaveHandler:

```
SWAPF STATUS_TEMP, 0 ; Swap STATUS_TEMP register into W
; (sets bank to original state)
MOVWF STATUS ; Move W into STATUS register
SWAPF W_TEMP, 1 ; Swap W_TEMP
SWAPF W_TEMP, 0 ; Swap W_TEMP into W
RETFIE ; Return from interrupt at point of interruption+1
```

```
; Interrupt Vector at 0004h. Execution starts here after an interrupt.
PSECT isrVect, class=CODE, delta=2
```

InterruptVector:

```
goto ISRHandler
```

Code 1-1 Timer2 Set up & function code

RA0 $\rightarrow$ Toggling in main

RA1 $\rightarrow$ diagnostic pin

Predicted Period: 20,011 μ s

11Tcy + 20,000 Tcy

Because of where I placed the clear of the timer 2, all the code between the Interrupt Vector & the clear adds to the total period

3Tcy $\rightarrow$ INT Detection
2Tcy $\rightarrow$ Goto ISRHandler
3Tcy $\rightarrow$ Context Save
3Tcy $\rightarrow$ BTF & goto Service
6Tcy $\rightarrow$ Servicing INT
6Tcy $\rightarrow$ Context Load & RETFIE

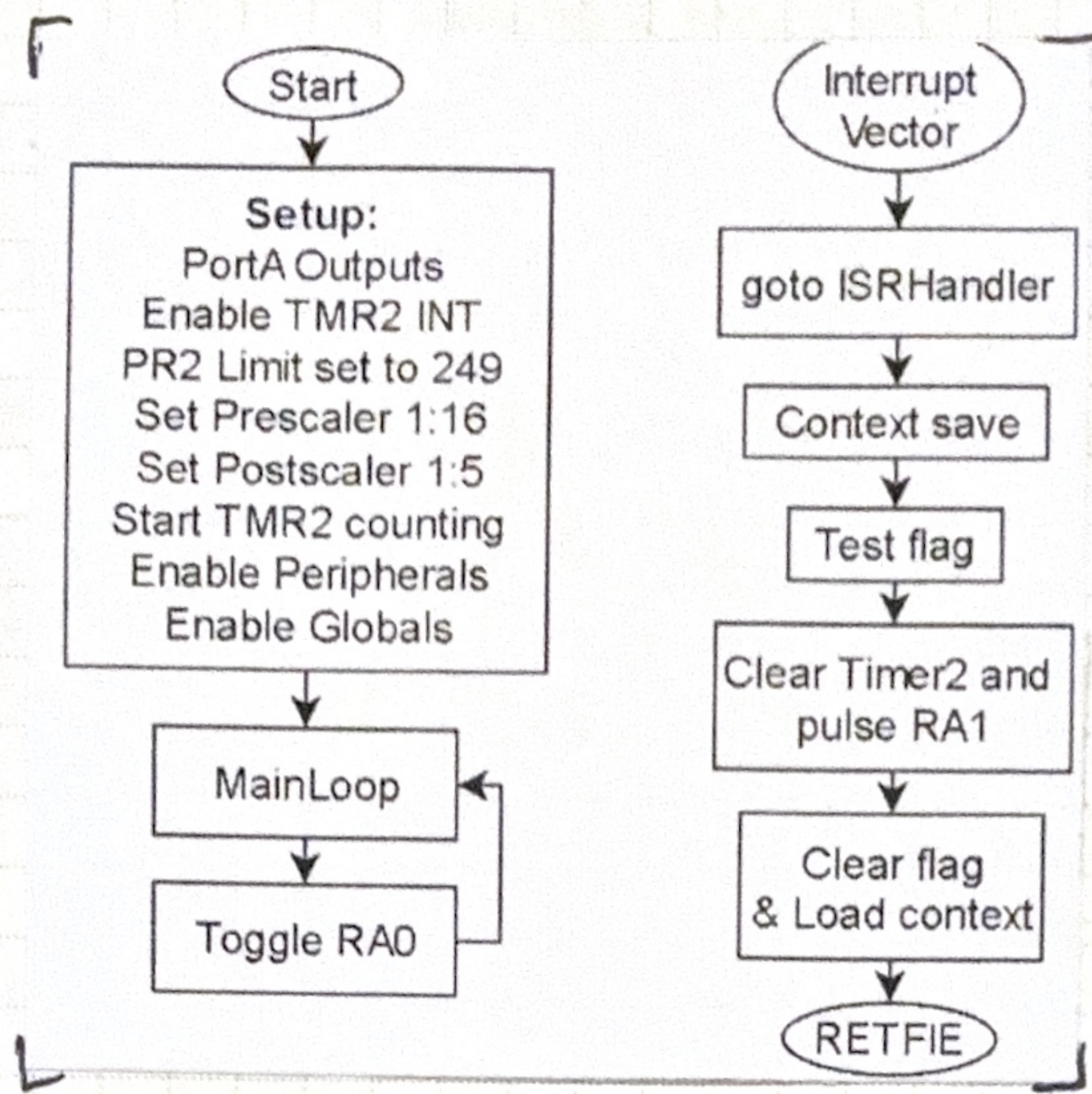
added Time

Clearing the count is effectively the timer period boundary, however it does ignore the 11 μ s of instruction cycles above it. But Clearing it here puts the beginning of the count as close to the rising edge of the diagnostic pin as possible

ART
9/29/26
191

Experiment 6 Timers & Event-Driven State

ART
9/29/24



Fe 1-1 based on
Code 1-1

In the Lab

Figure 1-1
Timer Diagnostic
Pulse Period
measurement

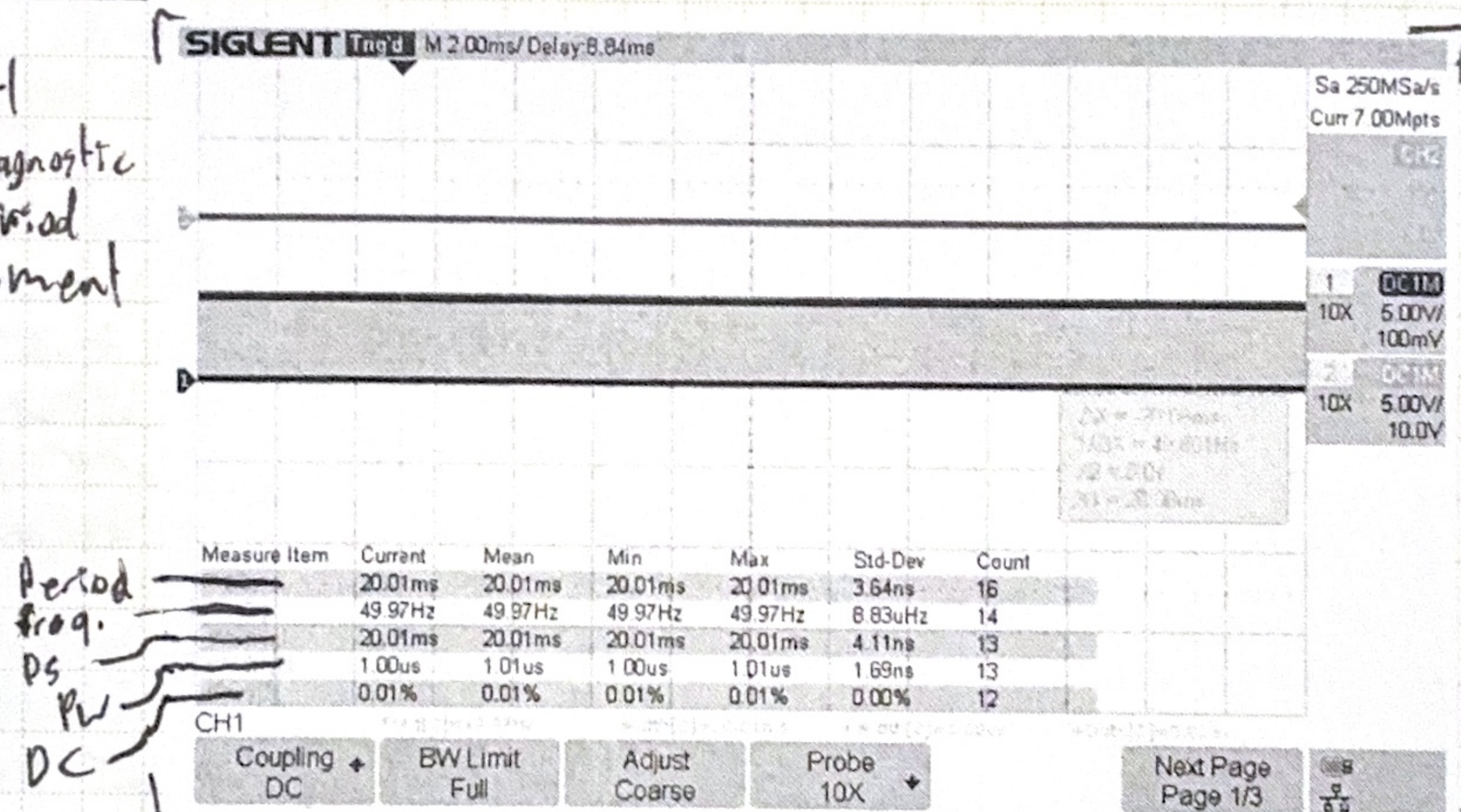


Figure 1-2
Diagnostic
pulse distance
from INT
measurement

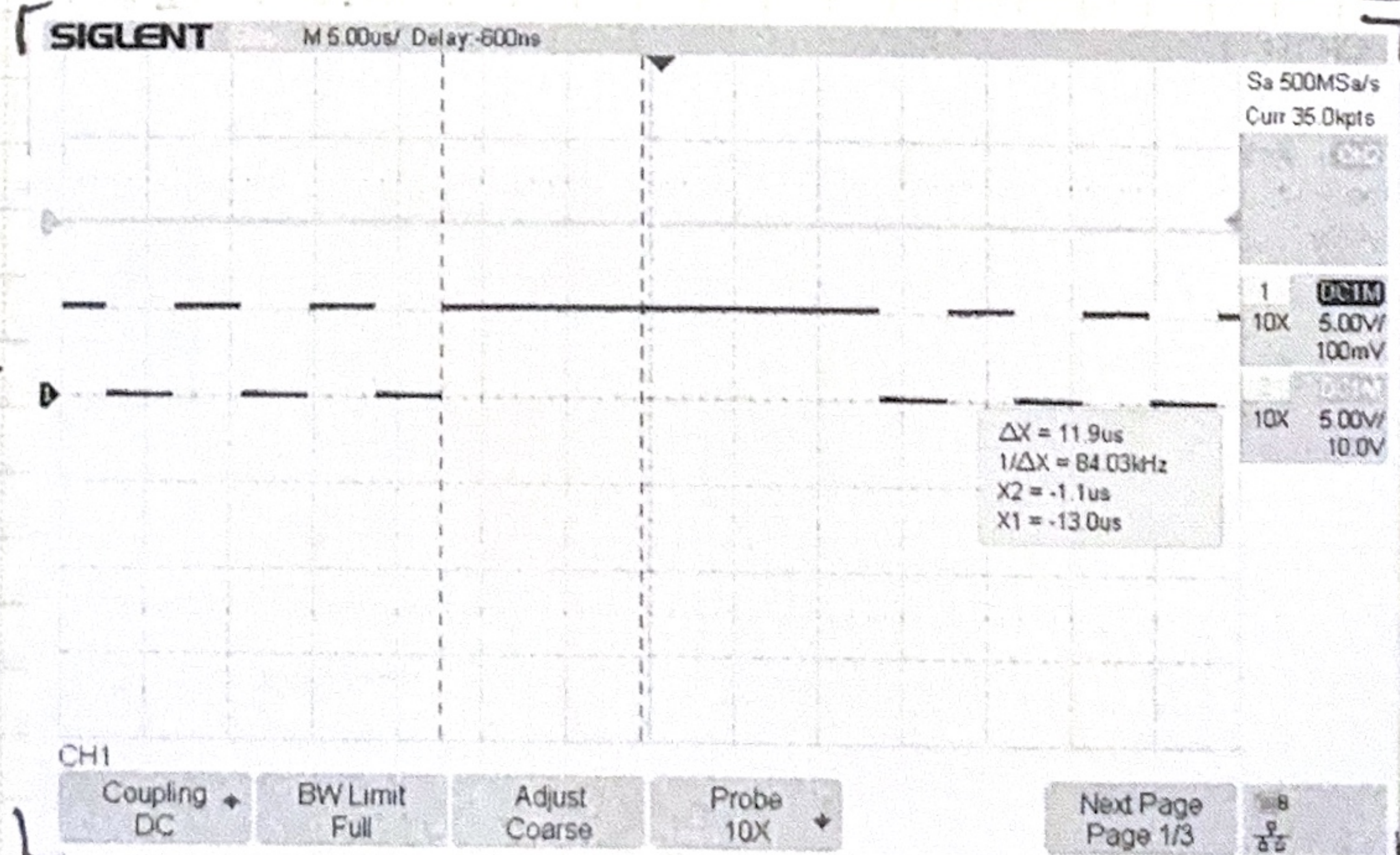
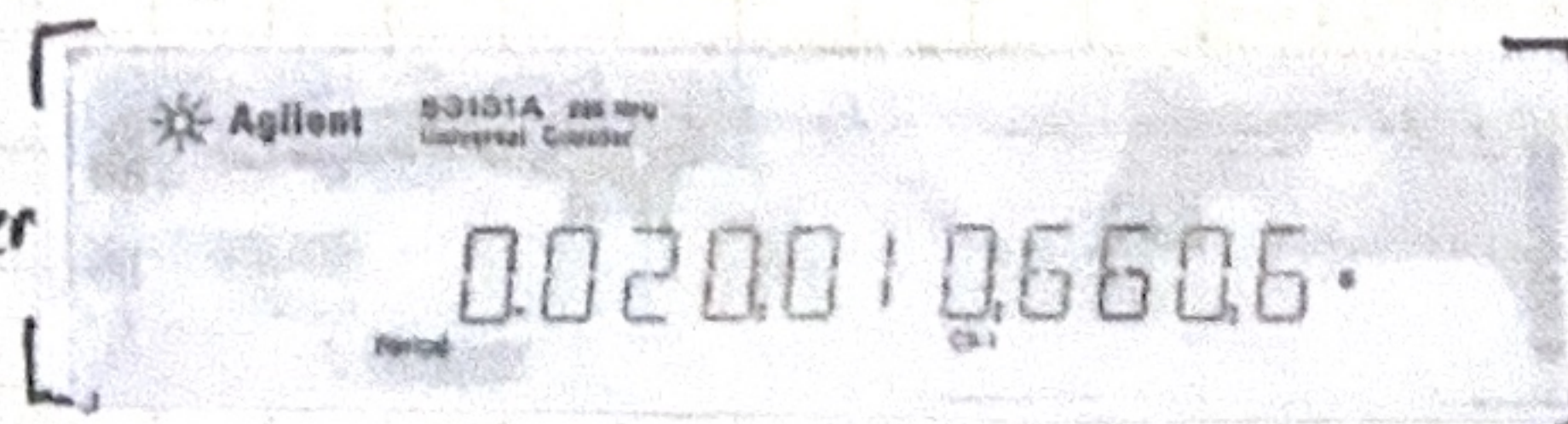


Figure 1-3
Period freq Counter
Diagnostic pulse



ART
9/29/24

Part 2 - Intersection State Machine LogicGoal

Build & demonstrate the traffic-light state machine without hardware timers. Use the External INT as a manual state advance event. Each valid external int increments a 4-bit state_count field in intersection_state. Use portB IOC to latch car-detection events. Main evaluates the packed state & decides whether the current traffic direction remains selected or begins a transition. On every iteration of Main, display the contents of intersection_state on portC so the byte can be observed on LEDs.

Required state register

Use a GPR named IntersectionState (GPR Address 0x72) Global GPR's

Register Map Table 2-1

Bit	7	6	5	4	3	2	1	0
Name	SC3	SC2	SC1	SC0	Transition	EW_Detected	NS_Detected	Direction

The upper nibble is a 4-bit state_count field & the lower nibble contains intersection flag bits

Field	State count 3:0	Direction	NS_Detected	EW_Detected	Transition
meaning	Current manual state machine count	0 = N/S direction 1 = E/W direction	Low-high event latched N/S	Low-high event latched E/W	0 = direction green 1 = direction yellow

function definitions Table 2-2Initial State

• State_Count = 0, Direction = 0 (N/S), Transition = 0, Car detection flags clear
main copies this to PortC

External INT State-Advance

Each valid external INT must increment the state_count bits only using a read-modify-write technique. The lower four flag bits must be unchanged.

PortB IOC Car Detection

Use two portB IOC's for: N/S car detected sets flag, E/W car detected sets flag

Once set, the flags remain set until the state machine clears it. Multiple IOC's on the same pin do nothing. The flag simply remains set.

PortC State Display

On every iteration of Main:

1. Copy the complete intersection_state byte to PortC
2. Then evaluate state_Count & the state flags
3. Update intersection_state only when the current state requires action

State Count Decisions

Main must examine state_count & decide what action or test is required

Table 2-3 state_count Behavior

State_Count	0, 1, 2	3	4
Behavior	do nothing	evaluate the Low byte & decide to remain or begin a transition	Finish an active Transition

Decision @ state_count = 3

When state_count = 3, evaluate the flags based on this Logic

N/S Green (Direction=0)			E/W Green (Direction=1)	
N/S detected	E/W detected	Action	N/S detected	E/W detected
1	0	remain N/S	0	1
0	1	set Transition	1	0
1	1	set Transition	0	1
0	0	Set Transition	0	0

Table 2-4 Logic rules

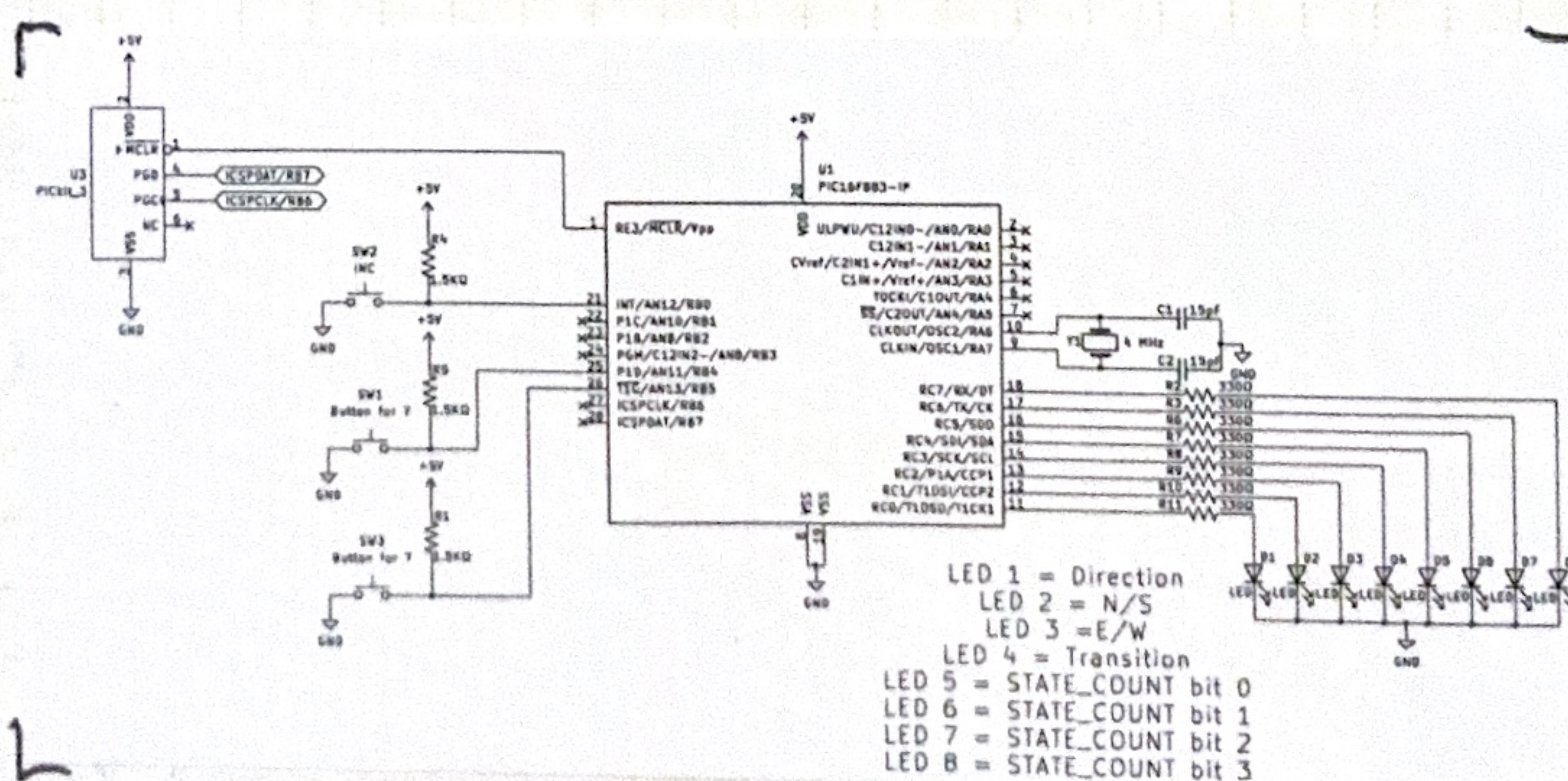
If you don't change direction Clear: state_count, N/S detected, E/W detected

If you change direction: Set Transition (if already set do Nothing)

① state_count = 4: toggle Direction, Clear transition, flags, state_count.

Before LabPrepare:

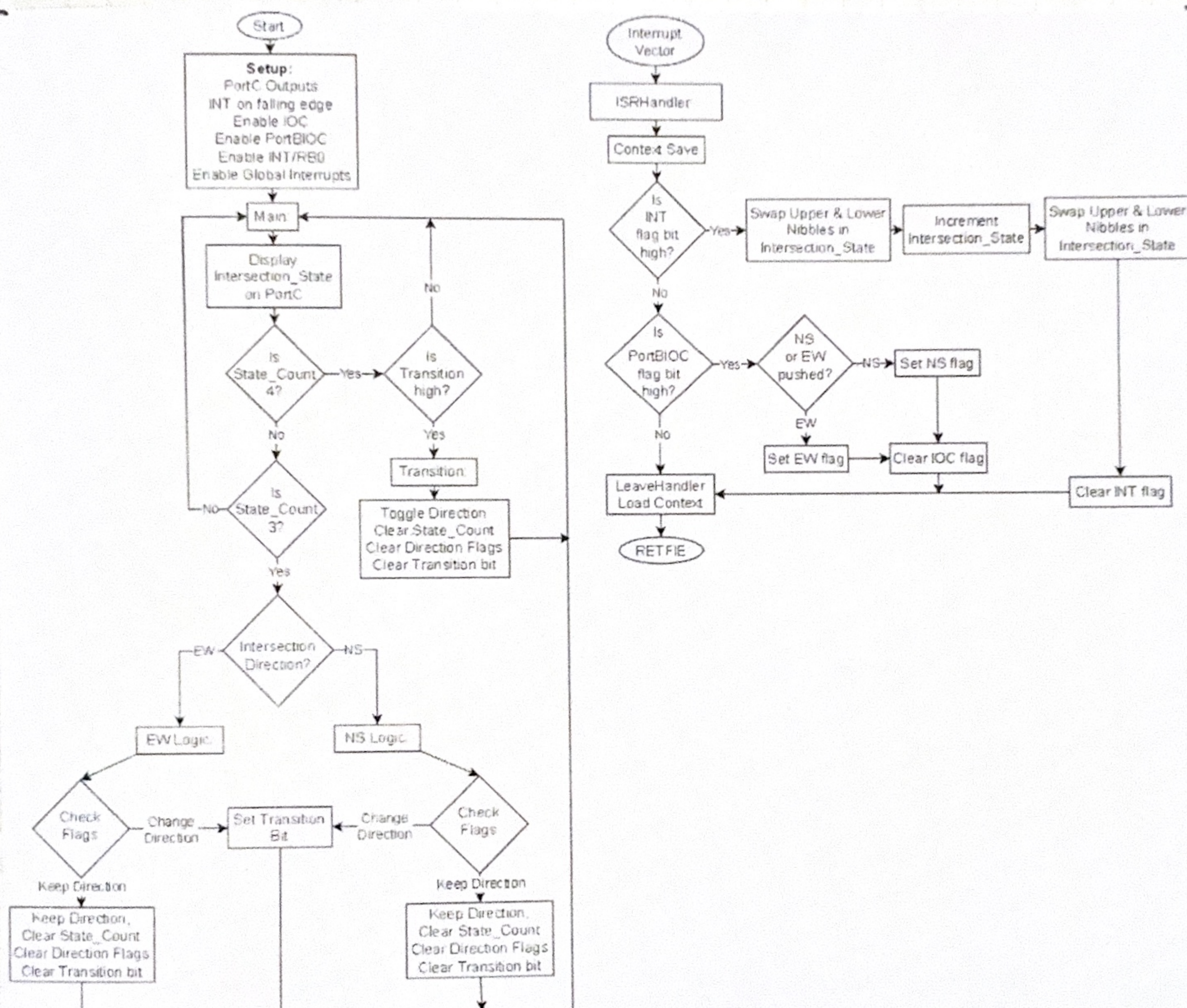
- external-interrupt input circuit & IOC circuit input
- External INT IOC SFR's
- Intersection - State register map
- Complete flowchart for every state_count value
- four flag truth table & simplification on decision
- source code

Figure 2-1
INT & IOC
inputs with
LED's on PortC

ART
9/30/26

* Only two Situations for the NS/EW flags cause a Unique situation. Therefore the simplest build is achieved by Observing which direction we start at, seeing if that direction's flag is up = stay same with every other possibility toggling.

Sum 0
change 0
Nothing



FC2-1 for code 2-1 & Code 2-2

* Because of the way the code is setup, & the function works. The SC3 will never tick high. This is not true for SC2 however its high tick happens too fast to be seen.

*

ART
9/30/26

```

Main:
    MOVF Intersection_State,W    ; Copy byte to W
    MOVWF PORTC                ; Display Intersection_State on PortC

Verify4:
    BTFSC Intersection_State,6
    goto Is4

Isnt4:
    BTFSS Intersection_State,4 ; Test SC0, if high test SC1
    goto Main                ; if low, return to main (already found 4)

    BTFSS Intersection_State,5 ; Test SC1, if high, a 3 is detected
    goto Main                ; if low, return to main (already found 4)

; By this point a 3 is detected
    BTFSS Intersection_State,0 ; Test which direction we're going
    goto NSLogic

EWLogic:
    BTFSC Intersection_State,1
    goto SetTransition
    BTFSC Intersection_State,2
    goto Remain
    goto SetTransition

NSLogic:
    BTFSS Intersection_State,1
    goto SetTransition
    BTFSS Intersection_State,2
    goto Remain
    goto SetTransition

SetTransition:
    BSF Intersection_State,3
    goto Main

Remain:
    BTFSS Intersection_State,0
    goto ToggleFromEW

    CLRF Intersection_State
    BSF Intersection_State,0
    goto Main

Is4:
    BTFSC Intersection_State,3 ; Test if we should transition
    goto Transition          ; If high, transition
    goto Main

Transition:
    BTFSC Intersection_State,0 ; Test which direction we're going
    goto ToggleFromEW

    CLRF Intersection_State
    BSF Intersection_State,0
    goto Main

ToggleFromEW:
    CLRF Intersection_State
    goto Main

```

} display Intersection_State
On port C
} Verify State_Count is on
Int 4

} If State_Count $\neq 4$, test if
it = 3. 0, 1, 2 don't do anything.
If State_Count = 3,
check which Direction it's
going & perform accordingly.

} If Direction = EW, EW flag = 1
Transition LED goes on off
& nothing changes

} Same for this for NS.
Keep in mind if both flags
are high or low, a transition
must occur

} set transition bit

} If no transition occurs,
the Direction must not
change, while everything else
in Intersection_State clears

} If State_Count = 4, test
transition bit & move accordingly

} Toggles Direction while
clearing everything else
in Intersection_State.

Code 2-1 Main Loop Intersection monitoring

ART

10/2/26

Experiment 6 Timers & Event-Driven State

ART
9/30/26

```
ISRHandler:
    MOVWF W_TEMP      ;Copy W to TEMP register
    SWAPF STATUS,0     ;Swap status to be saved into W
                        ;Swaps are used because they dont change the status bits
    MOVWF STATUS_TEMP  ;Save status to bank zero STATUS_TEMP register

    BTFSC INTCON,1      ;Test the INT external flag
    goto INTExternalService ;if the flag is up go to service
    BTFSC INTCON,0      ;Test the IOCPortB flag
    goto IOCPortBService  ;if the flag is up go to service
    goto LeaveHandler

INTExternalService:
    MOVF Intersection_State,W ; Read the Contents of the register
    MOVWF State_Count        ; Place the contents into changeable register
    SWAPF State_Count,1      ; Swap upper and lower nibbles for incrementing
    BCF State_Count,3        ; Clear the high Status_Count bit for safety
    INCF State_Count,1       ; Increment Status_Count
    SWAPF State_Count,0      ; Swap upper and lower nibbles to return
    MOVWF Intersection_State ; Restores Intersection_State

    BCF INTCON,1            ; Clear the INTExternal flag
    goto LeaveHandler

IOCPortBService:
    BTFSS PORTB,4           ; Was N/S pushed?
    BSF Intersection_State,1 ; Set the N/S flag
    BTFSS PORTB,5           ; Was E/W pushed?
    BSF Intersection_State,2 ; Set the E/W flag

    BCF INTCON,0            ; Clear the IOCPortB flag
    goto LeaveHandler

LeaveHandler:
    SWAPF STATUS_TEMP,0     ; Swap STATUS_TEMP register into W
                            ; (sets bank to original state)
    MOVWF STATUS            ; Move W into STATUS register
    SWAPF W_TEMP,1          ; Swap W_TEMP
    SWAPF W_TEMP,0          ; Swap W_TEMP into W
    RETFIE                  ; Return from interrupt at point of interruption+1
```

Interrupt flag
checks
& context save

Increments the
upper nibble of
Intersection_State

Detects IOC button
push to set either
N/S flag or E/W flag

Load context
& leave

Code 2-2 Interrupt Direction flags & State_Count
Incrementing

TJR
10-1-26

ART

10/2/26

ART
10/1/26
197

Part 3 - State Machine Intersection Timing

Goal

Use the Intersection state register to track & control Intersection Timing. The State_Count will now be incremented by a Timer ISR. Choose one timer interval & state count Range to allow a 1 second & 5 second delay to be represented. PortA will display the actual traffic-Light signals. Car detection is ignored in this section.

Required Timing

N/S Green	N/S Yellow	N/S Red	N/S Red
E/W Red	E/W Red	E/W Green	E/W yellow
5 sec	1 sec	5 sec	1 sec

Timer Driven State_Count

- Direction Flags & IOC interrupts are Disabled
- One hardware timer generates an interrupt that advances State_Count

5 & 1 Seconds must be achieved in full whole-number intervals of timer interrupts.

document Timer used & state_Count Map

Port A outputs

use portA to drive six LEDs representing normal traffic light signals

Main must decode Direction & Transition to direct the traffic lights

Before Lab

Prepare:

- Selected hardware timer & reasoning. Calculate timer period
- Calculate state_Count values for 5 & 1 seconds
- Timer SFR documentation (pg 306 & 307)
- Traffic light Schematic
- flowchart
- Source Code

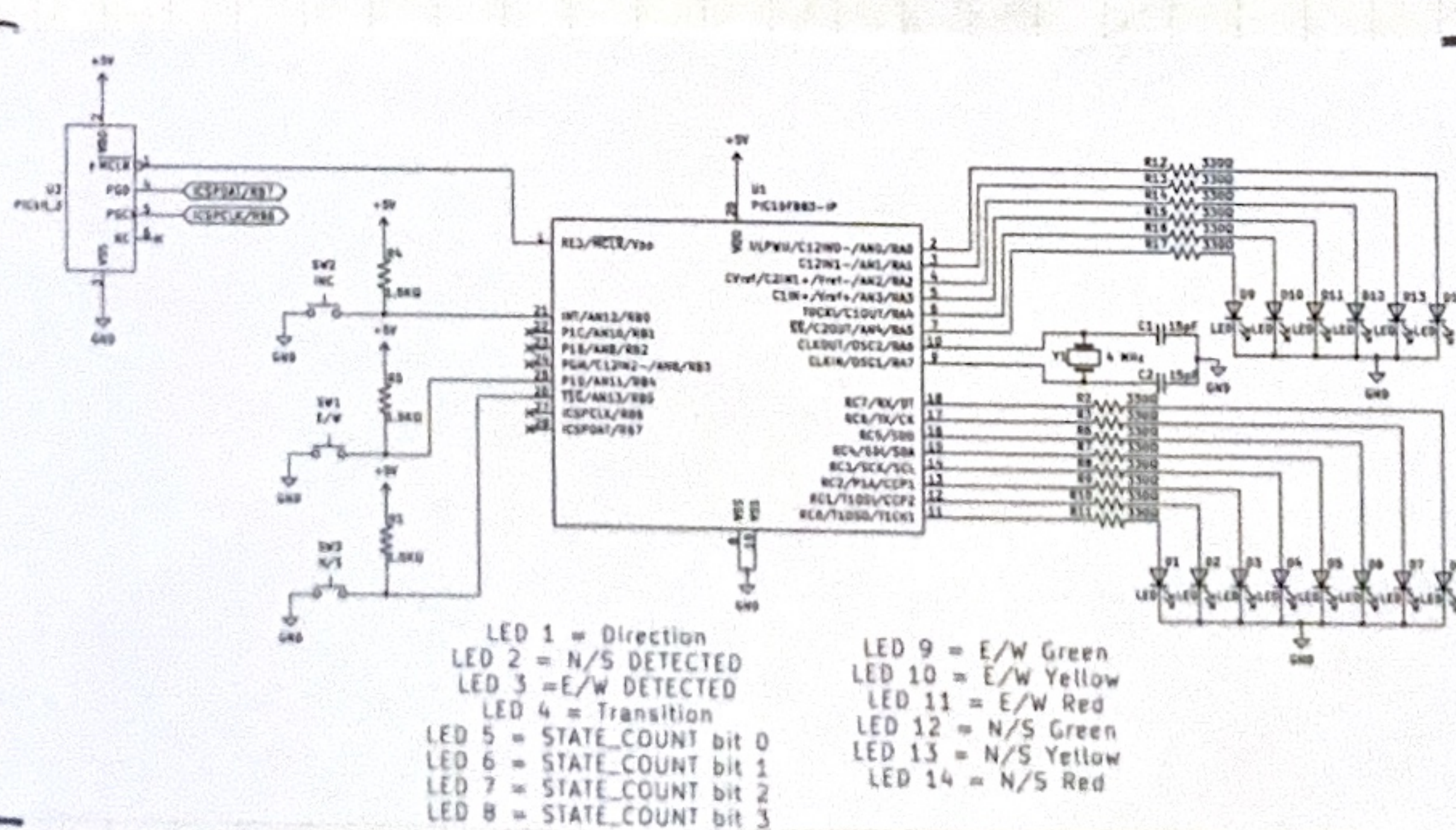


Figure 3-1

Full traffic
light display
circuit

* The timer I used was the Timer1 because it has the largest count. With two bytes of count its max is $2^{16} = 65,536$. That with a prescaler of 8 $\text{Max} = 524,288$.

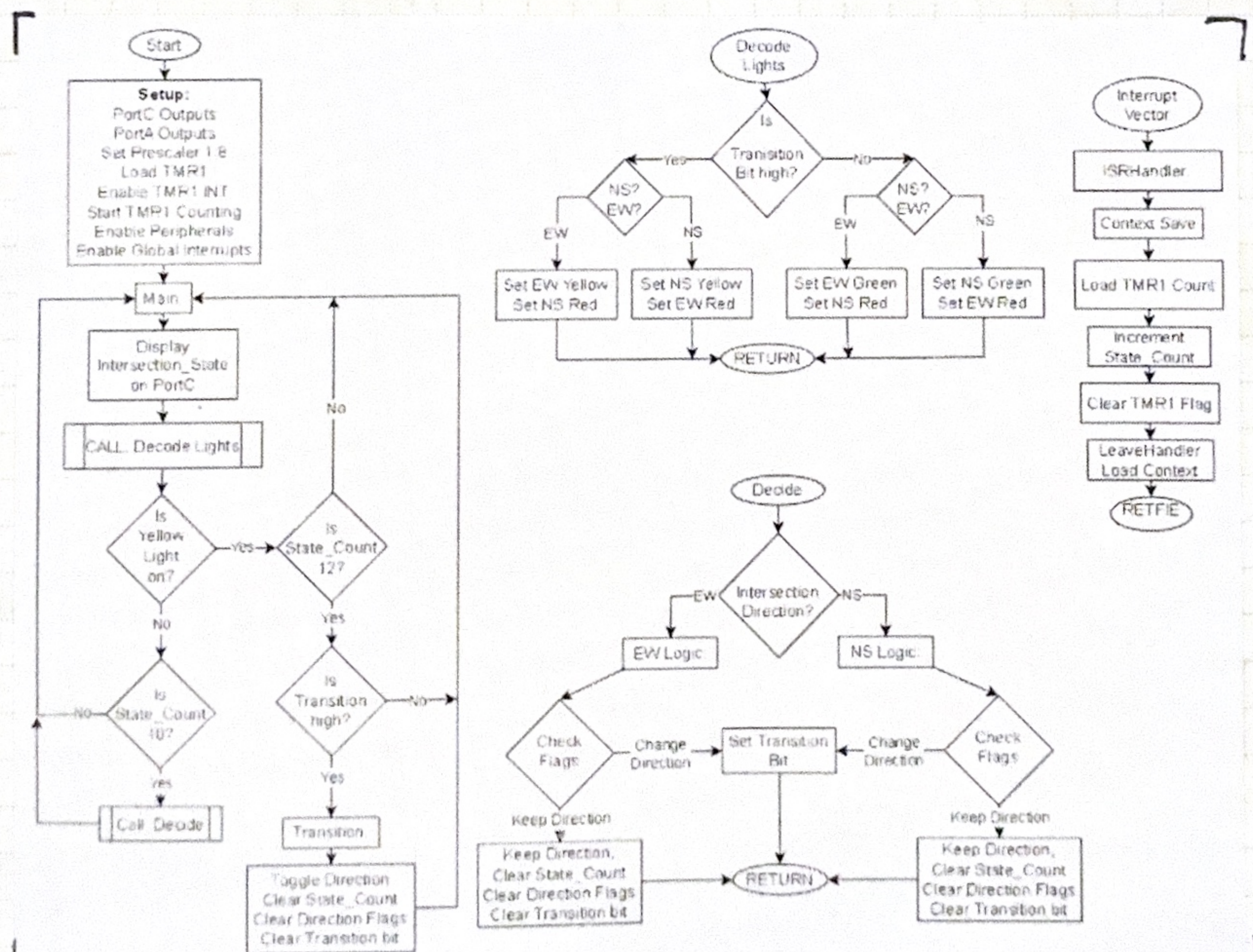
Period TMR1 Loading Calc: $.55 / 8 \mu s = 62,500$ $65,536 - 62,500 = 3,036$

Load 3,036 $\rightarrow$ 0X0BDC

State_Count	Time	Action
0 0 0 0	.0 sec	Waiting
0 0 0 1	.5 sec	on
0 0 1 0	1 sec	Green
0 0 1 1	1.5 sec	for
0 1 0 0	2 sec	Count=10
0 1 0 1	2.5 sec	
0 1 1 0	3 sec	
0 1 1 1	3.5 sec	
1 0 0 0	4 sec	
1 0 0 1	4.5 sec	
1 0 1 0	5 sec $\leftarrow = 10$	
1 0 1 1	.5 sec	
1 1 0 0	1 sec $\leftarrow = 12$	

* On startup the transition Bit is on so green light. Count up to ten for 5 sec, enable the transition Bit for 1 sec on a yellow light

Table 3-1 State_Count Values for Timing



FC 3-1 for Code 3-1

Experiment 6 Timers & Event-Driven State

ART
10/2/26

```

Main:
    MOVF Intersection_State,W    ; Copy byte to W
    MOVWF PORTC                ; Display Intersection_State on PortC

    CALL DecodeLights

    BTFSC Intersection_State,3    ; already in yellow?
    goto YellowWait

    BTFSS Intersection_State,7    ; green: wait for count 10
    goto Main
    BTFSS Intersection_State,5
    goto Main
    CALL Decide
    goto Main

YellowWait:
    BTFSS Intersection_State,6    ; count reached 12 (1 s of yellow)
    goto Main

Transition:
    BTFSS Intersection_State,0    ; Test which direction we're going
    goto GoEW                    ; If high Clear the register to change to NS

    CLRF Intersection_State
    goto Main

GoEW:
    CLRF Intersection_State      ; If low clear the Reg then set the direction bit
    BSF Intersection_State,0     ; Set bit to change to EW
    goto Main

Decide:
    BTFSC Intersection_State,3    ; is TRANSITION bit set low = skip
    RETURN                      ; dont do anything
    BTFSS Intersection_State,0    ; look which direction is green N/S=0, E/W=1
    goto NSLogic

EWLogic:
    BTFSC Intersection_State,1    ; Test the NS flag
    goto SetTransition          ; If high, will toggle so set bit 3
    BTFSC Intersection_State,2    ; Test the EW flag to check for Remain
    goto Remain                ; If high, Remain EW
    goto SetTransition          ; If low toggle so set bit 3

NSLogic:
    BTFSS Intersection_State,1    ; Test the NS flag
    goto SetTransition          ; If low, will toggle so set bit 3
    BTFSS Intersection_State,2    ; Test the EW flag to check for Remain
    goto Remain                ; If low, Remain NS

SetTransition:
    BSF Intersection_State,3      ; Set the transition bit
    RETURN

Remain:
    BTFSS Intersection_State,0    ; Check Direction we're going
    goto ToNS                  ; If low Clear the Reg to remain NS

ToEW:
    CLRF Intersection_State      ; If high Clear the Reg
    BSF Intersection_State,0     ; Then set the Direction bit to remain EW
    RETURN

ToNS:
    CLRF Intersection_State      ; Clear register to change to NS
    RETURN

DecodeLights:
    BTFSS Intersection_State,3    ; is TRANSITION yellow flag on
    goto GreenPhase            ; no we're in a green/red phase
    goto YellowPhase            ; yes we're in a yellow/red phase

GreenPhase:
    BTFSS Intersection_State,0    ; which way is green?
    goto SetNSGreen             ; bit was 0 N/S green
    goto SetEWGreen             ; bit was 1 E/W green

YellowPhase:
    BTFSS Intersection_State,0
    goto SetNSYellow
    goto SetEYYellow

SetNSGreen:
    MOVLW 0b00001100            ; RA2 on (N/S green), RA3 on (E/W red)
    goto StoreLights

SetEWGreen:
    MOVLW 0b00100001            ; PA0 on (N/S red), RA5 on (E/W green)
    goto StoreLights

SetNSYellow:
    MOVLW 0b00001010            ; RA1 on (N/S yellow), RA3 on (E/W red)
    goto StoreLights

SetEYYellow:
    MOVLW 0b00010001            ; RA0 on (N/S red), RA4 on (E/W yellow)
    goto StoreLights

StoreLights:
    MOVWF PORTA                ; Takes moved values and then displays on the LEDs
    RETURN
    
```

Code 3-1

Test for state count = 10

Test for state count = 12

Explained on pg 196

Table 3-1 Traffic Light

DIRECTION	TRANSITION	N/S lights	E/W lights
0	0	Green	Red
0	1	Yellow	Red
1	0	Red	Green
1	1	Red	Yellow

Truth table

Setup:

```

; add device setup here
BSF 0x03, 06      ;
BSF 0x03, 05      ; Set bank 3 active
CLRF 0x189        ; ANSELH analog/digital select portB pins as digital
CLRF 0x188        ; ANSEL analog/digital select portA pins as digital

BSF 0x03, 06      ;
BCF 0x03, 5       ; Set bank 2 active

BCF 0x03, 06      ;
BSF 0x03, 05      ; Set bank 1 active
CLRF 0x85         ; Trisa sets all portA pins as outputs
CLRF 0x87         ; Trisc sets all portC pins as outputs
BSF 0x8C, 0       ; PIE1 Enable Timer1 interrupt

```

```

BCF 0x03, 06      ;
BCF 0x03, 05      ; Set bank 0 active
CLRF 0x05         ; Clear port A
CLRF 0x07         ; Clear port C
MOVLW 0b00110000  ; Put binary register in W
MOVWF 0x10        ; TMR1 prescaler to 1:8
MOVLW 0xDC        ; Move count placement in W
MOVWF 0x0E        ; Move W into TMR1L
MOVLW 0x0B        ; Move count placement in W
MOVWF 0x0F        ; Move W into TMR1H

; Interrupt setup
CLRF 0x0B         ; Clear INTCON to disable all interrupts by default
                        ; Disable PortB interrupt on change
                        ; Disable External interrupts
BSF 0x10, 0       ; Begin TMR1 counting
BSF 0x0B, 6       ; Enable Peripheral Interrupts
BSF 0x8B, 7       ; Enable Global interrupts

BCF 0x03, 06      ;
BCF 0x03, 05      ; Set bank 0 active
goto Main

```

ISRHandler:

```

MOVWF W_TEMP      ; Copy W to TEMP register
SWAPF STATUS, 0   ; Swap status to be saved into W
                        ; Swaps are used because they don't change the status bits
MOVWF STATUS_TEMP ; Save status to bank zero STATUS_TEMP register

BTFSC PIR1, 0     ; Test TMR1 flag bit
goto TMR1Service
goto LeaveHandler

```

TMR1Service:

```

MOVLW 0xDC        ; Move count placement in W
MOVWF 0x0E        ; Move W into TMR1L
MOVLW 0x0B        ; Move count placement in W
MOVWF 0x0F        ; Move W into TMR1H

MOVWF Intersection_State, W ; Read the Contents of the register
MOVWF State_Count         ; Place the contents into changeable register
SWAPF State_Count, 1      ; Swap upper and lower nibbles for incrementing
INCF State_Count, 1       ; Increment Status_Count
SWAPF State_Count, 0      ; Swap upper and lower nibbles to return
MOVWF Intersection_State  ; Restore Intersection_State

BCF PIR1, 0             ; Clear TMR1 flag bit
goto LeaveHandler

```

LeaveHandler:

```

SWAPF STATUS_TEMP, 0 ; Swap STATUS_TEMP register into W
                        ; (sets bank to original state)
MOVWF STATUS         ; Move W into STATUS register
SWAPF W_TEMP, 1      ; Swap W_TEMP
SWAPF W_TEMP, 0      ; Swap W_TEMP into W
RETFIE               ; Return from interrupt at point of interruption+1

```

Code 3-2

Interrupt Handler
& Setup CodeTJR
10-2-26

Good

Add one new feature: Use a portB input as a RUN/DEBUG selector

- RUN: Select hardware timer advances State Count
- DEBUG: External INT

Choose which Logic Level represents RUN & Debug. Changing modes should not affect the Intersection State Register or the signal lights. The detection IOC flags should work for either

Prepara:

- Part 2 state machine logic, truth table, detection inputs (pg 193, 194, 195)
- Part 3 timer Configuration, Calculations, state map (pg 198, 199)
- RUN/Debug Selector circuit
- Flowchart
- Source code

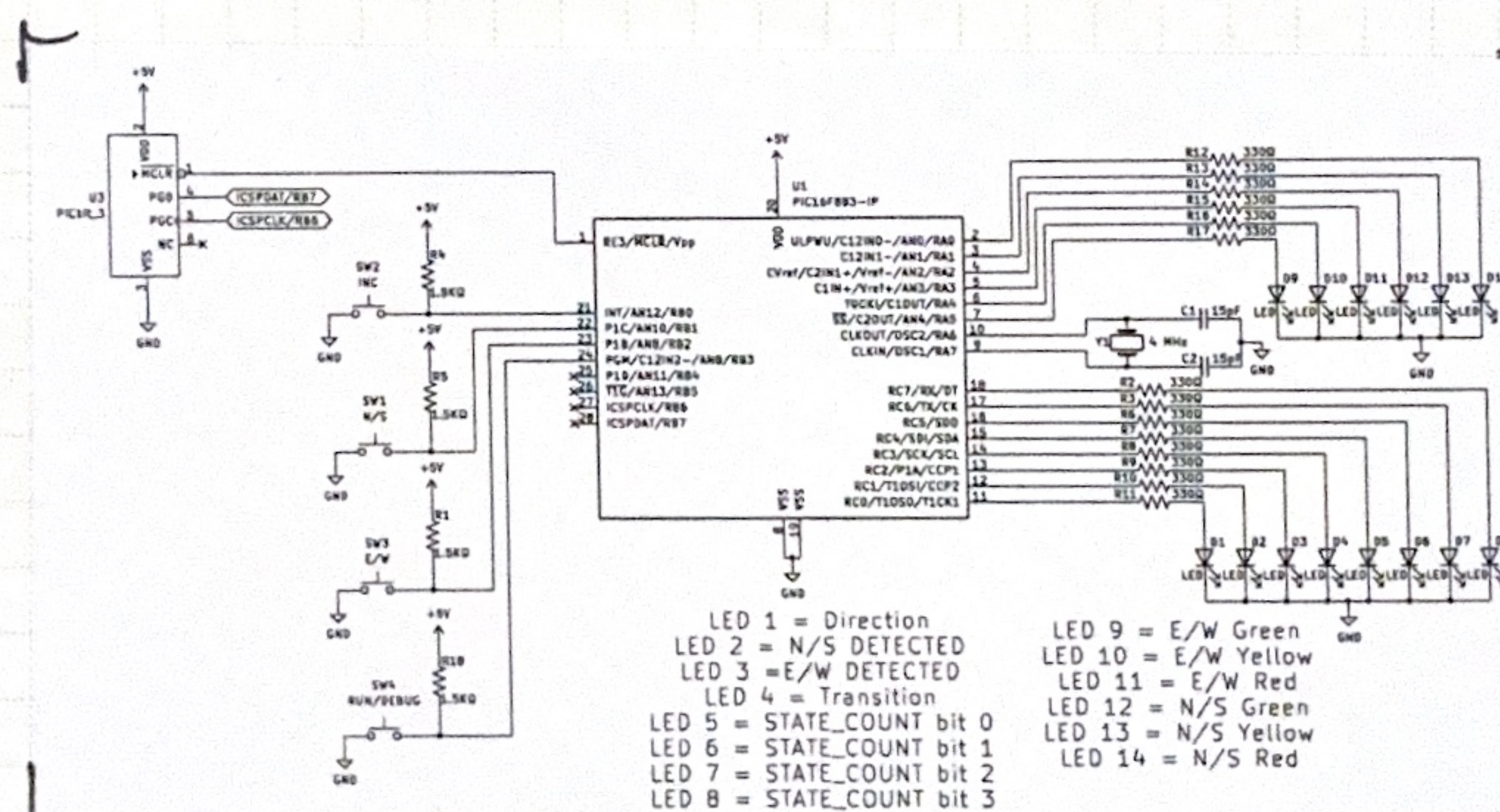


Figure 4-1 added RUN/DEBUG button

ART

10/2/20

Experiment 6 Timer & Event-Driven State

AKI
10/2/26

Mode:
BCF INTCON, 7
BTFSC PORTB, 3 ;bit set then DEBUG, matching your original polarity
goto Debug

Run:
BCF INTCON, 4 ;INTE off - button can't move the count in RUN
BCF INTCON, 1 ;clear any stale INTF
BCF PIR1, 0 ;clear any stale TMR1IF before starting fresh
BSF T1CON, 0 ;TMR1ON - timer actually counting
goto ModeDone

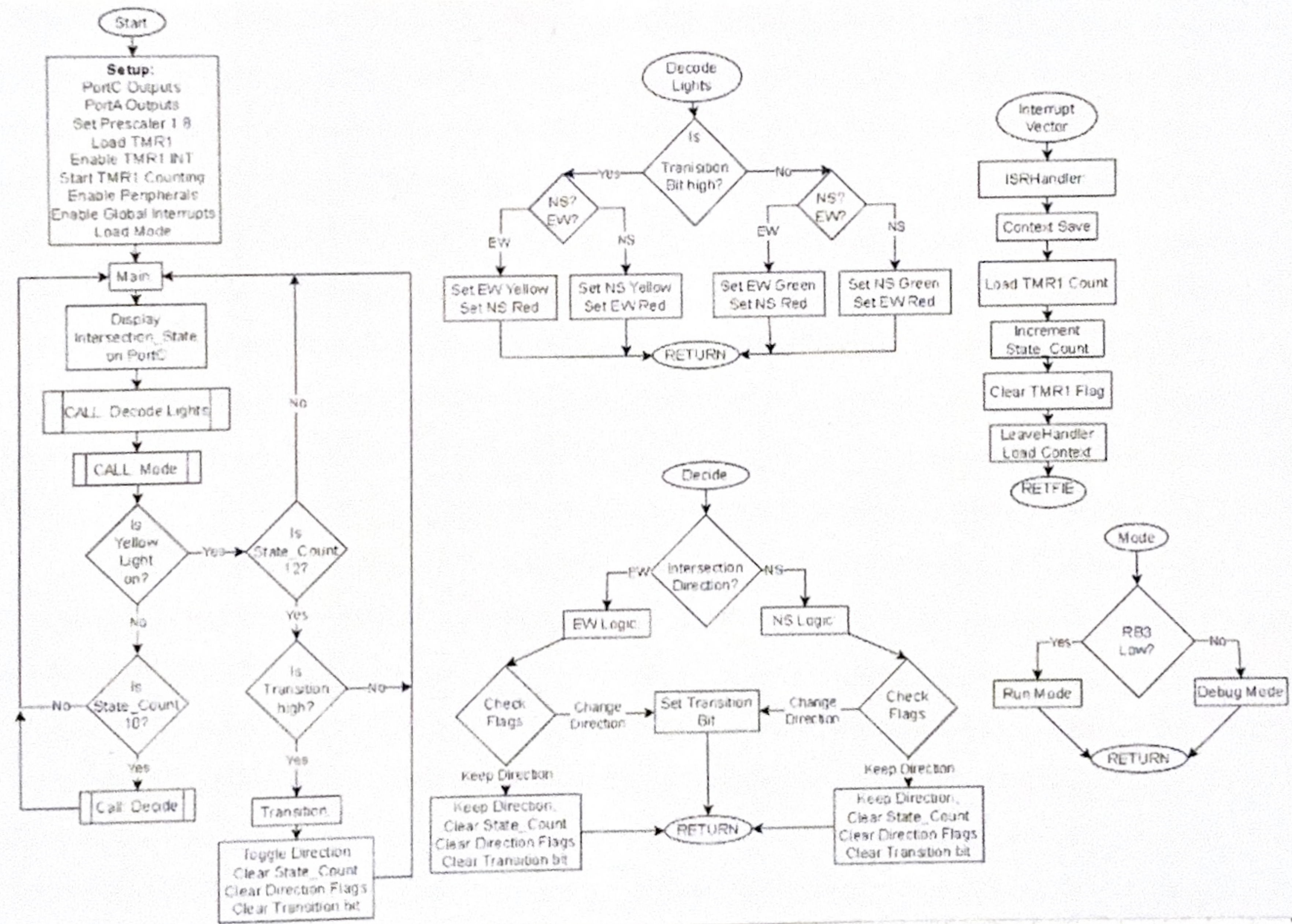
Debug:
BCF T1CON, 0 ;stop Timer1 outright - its flag can't set while stopped
BCF PIR1, 0 ;clear any stale TMR1IF
BCF INTCON, 1 ;clear any stale INTF from before the switch
BSF INTCON, 4 ;INTE on - button can move the count in DEBUG

ModeDone:
BSF INTCON, 7
RETURN

Code 4-1

Selector for RUN/DEBUG mode

* called in main



FC 4-1 for code 4-1

TJR
10-2-26

* All other code is identical, Only added call to toggle which interrupts are on

Conclusion

This Lab Experiment explored how Timers work & how they can be implemented to drive event states. The bulk of difficulty was placed mostly in creating the logical situation of an intersection in clean reusable code.

AKI
10/2/26